Options agent labs: levels 3 and 5

A small model works option-pricing and hedging problems it cannot solve in one shot. **The checker does the work**: it is exact, cheap, and never needs to be told the answer. Same loop as the heat-rod project, new problems.

```
controller   poses a problem                          level3_barrier.py /
level5_hedge.py
generator    the model proposes N answers             any OpenAI-compatible
server (Qwen3-8B on vLLM)
checker      grades each answer; grade = reward       the same two files
tool         level 3: calculator (COMPUTE:)           tool_calc.py
             level 5: simulator  (SIM:)               level5_hedge.py +
hedgesim.py
loop         best answer's feedback -> next prompt    agent.py
```

No data to import. Every problem comes from a seeded generator with a hidden ground truth.

The two levels

Level 3: barrier options. Write the pricing *formula* for a knock-out or knock-in call.

Sub	Problem	The trap
3.1	down-and-out call, no dividends, `H < K`	pricing it as a vanilla call
3.2	down-and-in call	returning the knock-out; needs in-out parity
3.3	down-and-out call with dividend yield, `H` above *or* below `K`	the closed form changes shape when `H > K`

Reward: 0.4 formula matches the true price at hidden test points, 0.2 correct when spot sits on the barrier, 0.2 correct limit when the barrier is far away, 0.2 stays within no-arbitrage bounds.

Level 5: hedging a short call with transaction costs. Minimise `J = E[cost] + lam * std(P&L)` on simulated paths.

Sub	The model chooses	Notes
5.1	an interval `m` (look at the hedge every m-th step)	one integer
5.2	a no-trade band `b`	one number
5.3	a band *formula* `band(gamma, S, tau)`	tiers are stricter: a constant band cannot reach 1.0

Reward, 0.2 each: valid answer / beats not hedging / beats the daily full hedge / within 10% of the best achievable / within 3% of it (5.3: 5% and 1%).

What is in this folder

File	What it is
`optcheck.py`	shared core: safe formula evaluator (AST whitelist), Black-Scholes, barrier reference, independent Monte Carlo
`level3_barrier.py`	generator, checker, selftest for level 3
`hedgesim.py`	the hedging simulator and the reference grid-search
`level5_hedge.py`	generator, checker, SIM tool, selftest for level 5
`tool_calc.py`	the level 3 calculator
`agent.py`	the loop (sampling, tools, feedback, JSONL log, offline mode)
`test_plumbing.py`	proves the tool loop works end to end with a scripted model, no server needed
`report.py`	summarises `attempts.jsonl` by tag
`run_all.sh`	selftest / offline / real / experiments

Needs `numpy` and `scipy`. A real run also needs `httpx`. Offline mode and all tests need no server.

Run it

1. Prove the checkers (no model, about 2 minutes). Do this first; a wrong checker makes every score meaningless.

```
bash
```

```
./run_all.sh selftest
```

You want four lines saying `SELFTEST PASSED` / `PLUMBING PASSED`. What each proves:

- `optcheck.py`: the barrier formula agrees with an independent Monte Carlo on 12 cases (including `H > K` and dividends), and the evaluator rejects unsafe input.
- `level3_barrier.py`: the exact formula scores 1.0 on every generated problem; every trap formula scores below 1.0 and the feedback names the failing region.
- `level5_hedge.py`: the simulator obeys physics (zero mean P&L without costs, error falls like 1/sqrt(steps), cost linear in c, optimal band widens with c), and no problem is too easy.
- `test_plumbing.py`: tool requests are parsed, run, fed back, graded and logged.

2. Try the loop with a fake model (numbers meaningless, never report them):

bash

```
./run_all.sh offline
```

3. Real runs. Start your model server (on the seat pod: `./serve.sh` from the repo root, wait for `READY`), then in a second terminal in the same pod:

bash

```
export OPT_BASE_URL=http://localhost:8000/v1     # the pod already sets HEATROD
cd /workspace/projects/02-options-agent          # wherever you copied this fol
./run_all.sh real                                # 3 runs x 2 levels, in the ba
tail -f run-l3-1.log
python report.py attempts.jsonl                  # when they finish
```

Runs go to the background with `nohup`, so a dropped `kubectl exec` does not kill them. Reconnect and `tail -f` the log. `pgrep -af agent.py` shows whether one is still running.

Single problems, by hand:

bash

```
python level3_barrier.py --show 1 --answer       # the prompt, and a correct an
python agent.py --level 3 --sub 1
python agent.py --level 5 --sub 2 --samples 4 --rounds 4
```

4. The experiments (the clearest evidence in the project):

```bash
./run_all.sh experiments    # no tools / score-only feedback / no hint, 3 runs
python report.py attempts.jsonl
```

Flag	Question it answers
`--no-tools`	what does the calculator / simulator buy?
`--feedback score`	how much slower is the agent when the checker says only *how much*, not *what*?
`--feedback none`	does it improve at all without feedback?
`--hint 0`	can it find the reflection principle unaided?

One run is not a result. At four samples per round the numbers move between runs. Report how many runs you did; `report.py` prints the count.

Notes and known limits

- **The first check of each level-5 problem takes 15-45 seconds** while the reference optimum is computed (grid search on 20,000 held-out paths). It is cached afterwards, so only the first round of a problem is slow.
- **Level 5 is graded on different paths than the SIM tool shows.** The tool uses 2000 practice paths, the checker 20,000 held-out ones, so tuning to noise does not pay.
- **On 5.3 structure helps only a little**: the best state-dependent band is about 2% better than the best constant band in the default problem. The tiers are set so that a constant band tops out at 0.8, but do not expect a dramatic gap.
- **3.3 is meant to be hard.** If a model solves it, the next step is a harder exotic.
- **Level 5 uses a mean-std objective**, not exponential utility, so the Whalley-Wilmott constant is not exactly right; the cube-root scaling is the part that carries over.
- **The reference solver for level 5 is a grid search**, so "within 3% of best achievable" means within 3% of the best grid point.
- Models can read this repo's source. If you point a model at the folder, it can read the answers.

Where to take it

- **Add a trainer.** `attempts.jsonl` is already prompt, answer, reward.
- **Harder products.** Arithmetic Asian (Monte Carlo reference with a standard-error tolerance), or American options priced on a lattice.
- **Calibration.** Fit Heston parameters to a synthetic smile (the original level 4 idea).

- **Make the checker worse on purpose** (`--feedback score`) and plot the slowdown.